STM32WL55JC1

Project 02 - Debugging con GDB + ST-LINK

Cortex-M4 | reset e stack | disassembly Thumb | registri memory-mapped

Obiettivo: osservare dal vivo il firmware del Project 01, dal reset fino all'accensione del LED verde su PB9, senza HAL e senza RTOS.

Documento di laboratorio - 4 ottobre 2026. I valori riportati provengono dall'ELF corrente e dagli output GDB raccolti durante la sessione. L'accensione del LED è stata confermata sulla scheda.

1. Obiettivo e risultato

Il Project 01 ha costruito il percorso dal reset al pin fisico. Il Project 02 lo ha reso osservabile: il processore è stato fermato, le istruzioni sono state eseguite singolarmente e i registri sono stati letti prima e dopo le scritture hardware.

Il firmware applicativo non è stato modificato. Sono stati aggiunti OpenOCD, una configurazione per la scheda e una guida GDB. Il core usato è CPU1, Cortex-M4; il Cortex-M0+ resta fuori dal laboratorio.

Risultati verificati: ingresso in Reset_Handler e main, evoluzione di PC/SP/LR, salvataggio di r7 sullo stack, abilitazione del clock GPIOB, configurazione di PB9 e accensione del LED dopo la scrittura su ODR. Sono stati verificati anche i breakpoint hardware su Reset_Handler e sull'ingresso esatto di main.

2. Strumenti e catena di debug

```
Mac: arm-none-eabi-gdb
          |
          | protocollo remoto GDB, localhost:3333
          v
        OpenOCD
          |
          | USB
          v
     ST-LINK-V3E
          |
          | SWD
          v
   STM32WL55 - Cortex-M4
```

Elemento	Ruolo
arm-none-eabi-gdb 17.2	Comandi, simboli, sorgente, registri e memoria
OpenOCD 0.12.0	Server remoto e controllo del target tramite ST-LINK
ST-LINK-V3E	Interfaccia USB/SWD integrata sulla Nucleo
firmware.elf	Codice, indirizzi, simboli e informazioni di debug
firmware.bin	Immagine binaria priva dei simboli di debug
-O0 e -g3	Build leggibile con informazioni di debug

La connessione reale ha riconosciuto ST-LINK V3J17M11, tensione target circa 3,26 V e Cortex-M4 r0p1. OpenOCD ha rilevato 6 breakpoint hardware e 4 watchpoint; i watchpoint non sono stati usati in questo esercizio.

3. Configurazione OpenOCD

Il file openocd.cfg nella radice del repository contiene:

```
source [find interface/stlink.cfg]
transport select hla_swd
set DUAL_CORE 0
source [find target/stm32wlx.cfg]
stm32wlx.cpu0 configure -event reset-init {}
adapter speed 500
```

La configurazione seleziona ST-LINK via SWD e il solo Cortex-M4. L'evento reset-init del file target standard viene svuotato perché normalmente aumenta la frequenza MSI per la programmazione: qui

interessa preservare la configurazione di clock dopo reset.

Durante il laboratorio OpenOCD ha segnalato: "Unable to match requested speed 500 kHz, using 200 kHz". È la velocità della connessione di debug scelta dall'adattatore, non la frequenza del Cortex-M4. La sessione ha funzionato regolarmente.

4. Avvio della sessione

Collegare la porta USB ST-LINK della scheda con un cavo dati e chiudere altre connessioni attive al debugger. Aprire due terminali nella radice del progetto.

Primo terminale:

```
make
openocd -f openocd.cfg
```

Lasciare OpenOCD attivo, in ascolto sulla porta GDB 3333. Secondo terminale:

```
arm-none-eabi-gdb firmware.elf
```

Inserire al prompt GDB una riga alla volta, premendo Invio dopo ogni comando:

```
target extended-remote localhost:3333
monitor reset halt
maintenance flush register-cache
```

Nella sessione il collegamento iniziale ha trovato il vecchio firmware già nel while finale. Il reset con halt ha riportato la CPU all'inizio del programma. Il comando maintenance forza GDB a rileggere i registri, evitando valori rimasti nella cache dopo un reset impartito tramite monitor.

Se l'immagine sulla scheda non corrisponde alla build, programmarla da GDB con la sequenza seguente; load scrive la Flash. Durante la verifica preliminare compare-sections aveva già confermato la corrispondenza di .isr_vector e .text.

```
monitor reset halt
load
monitor reset halt
maintenance flush register-cache
```

5. Vector table e stato dopo reset

```
x/2wx 0x08000000
info registers pc sp lr
x/i $pc
```

Output osservato:

```
0x08000000:  0x20010000  0x08000009
pc           0x08000008  <Reset_Handler>
sp           0x20010000
lr           0xffffffff
=> 0x08000008 <Reset_Handler>: bl 0x08000010 <main>
```

La prima word della vector table fornisce il valore iniziale di MSP. La seconda contiene il reset vector: il bit 0 indica lo stato Thumb, mentre l'indirizzo dell'istruzione è 0x08000008. Il processore non esegue codice all'indirizzo dispari 0x08000009.

SP punta inizialmente appena oltre la RAM del progetto, che termina a 0x2000ffff. Lo stack cresce verso indirizzi inferiori. Dopo reset, in questa applicazione SP è MSP. LR vale 0xffffffff: non rappresenta ancora un indirizzo di ritorno prodotto da una chiamata.

6. Registri Cortex-M4 e comandi di lettura

Registro	Significato nel laboratorio
PC	Punto di esecuzione; la freccia GDB mostra la prossima istruzione
SP / MSP	Stack pointer attivo, Main Stack Pointer in questo caso
LR	Indirizzo di ritorno salvato da BL
r0-r12	Registri generali usati dal compilatore
r7	Riferimento al frame di main nella build osservata
xPSR	Stato, flag e numero dell'eccezione corrente

OpenOCD ha riportato xPSR = 0x01000000 e current mode: Thread. Il bit T è impostato; il numero dell'eccezione corrente è zero. Il laboratorio non ha ancora introdotto interrupt o exception handler.

```
info registers
info registers pc sp lr r7
x/i $pc
x/6i $pc
x/wx 0x5800004c
disassemble /r main
```

In x/2wx: x significa esamina memoria, 2 è il numero di elementi, w seleziona word da 4 byte e x il formato esadecimale. In x/i, i seleziona il disassembly. Leggere una periferica dal debugger è comunque un accesso hardware: nel laboratorio sono stati letti registri RCC/GPIO, senza estendere questo uso a registri con effetti collaterali di lettura.

7. BL: ingresso in main e link register

Da Reset_Handler è stato eseguito:

```
stepi
info registers pc sp lr
x/i $pc
```

Output osservato:

```
pc  0x08000010 <main>
sp  0x20010000
lr  0x0800000d
=> 0x08000010 <main>: push {r7}
```

BL significa Branch with Link. Salta a main e registra il ritorno in LR; non spinge automaticamente il ritorno sullo stack. L'istruzione successiva alla BL è a 0x0800000c e LR contiene 0x0800000d, con il bit Thumb impostato. main non ritorna perché termina in un loop infinito.

8. Prologo di main e stack

Prima del push, r7 valeva zero. Dopo un singolo stepi:

```
pc  0x08000012
sp  0x2000fffc
r7  0x00000000
```

La lettura x/wx $sp ha restituito 0x00000000 a 0x2000fffc. push {r7} ha quindi decrementato SP di 4 byte e salvato in RAM il vecchio valore di r7, lasciando il registro invariato.

La prossima istruzione era:

```
add r7, sp, #0
```

Eseguendola con stepi, r7 è diventato 0x2000fffc, uguale a SP. Il compilatore usa questo registro come riferimento al frame della funzione nella build -O0 osservata. Questo prologo dipende dalla build e non va assunto identico per ogni programma C.

Un'istruzione assembly mostrata da GDB non va digitata come comando: add r7, sp, #0 ha prodotto un errore di comando ambiguo. Il comando che la fa eseguire è stepi.

9. RCC: indirizzo, valore e read-modify-write

Stato iniziale: RCC_AHB2ENR a 0x5800004c valeva 0x00000000.

```
0x08000014: ldr   r3, [pc, #52]
0x08000016: ldr   r3, [r3, #0]
0x08000018: ldr   r2, [pc, #48]
0x0800001a: orr.w r3, r3, #2
0x0800001e: str   r3, [r2, #0]
```

La costante a 0x0800004c nella Flash valeva 0x5800004c. La prima LDR carica questo indirizzo in r3 dalla literal pool; la seconda usa l'indirizzo per leggere RCC. Nelle LDR relative a PC, l'indirizzo effettivo è quello annotato dal disassembler: non si calcola sommando semplicemente l'offset al PC mostrato da GDB.

Dopo l'istruzione	Stato osservato
LDR a 0x08000014	r3 = 0x5800004c, indirizzo RCC
LDR a 0x08000016	r3 = 0, contenuto RCC
LDR a 0x08000018	r2 = 0x5800004c
ORR a 0x0800001a	r3 = 2, RCC ancora 0
STR a 0x0800001e	RCC = 2, GPIOBEN impostato

Questo è il read-modify-write di RCC_AHB2ENR |= (1U << 1). La modifica nella CPU diventa una modifica hardware soltanto con STR. Il valore 2 corrisponde alla maschera 1 << 1.

10. Read-back dopo l'abilitazione del clock

La riga (void)RCC_AHB2ENR è stata eseguita con due stepi:

```
0x08000020: ldr r3, [pc, #40]
0x08000022: ldr r3, [r3, #0]
```

Al termine, PC valeva 0x08000024 e r3 valeva 2. La lettura effettiva segue la scrittura di abilitazione prima degli accessi al GPIO. Il cast a void scarta il risultato; volatile preserva l'accesso hardware. volatile non rende atomica una sequenza read-modify-write.

I registri GPIO sono stati osservati dopo l'abilitazione e il read-back: letture con clock disabilitato non sono state usate come riferimento per la configurazione del pin.

11. MODER: PB9 da analogico a output

```
x/wx 0x48000400
p/x ((*(unsigned int *)0x48000400 >> 18) & 3)
```

Il valore iniziale di GPIOB_MODER era 0xffffffebf e il campo MODE9 valeva 3, cioè 11 binario (analogico). Il campo occupa i bit 19:18.

Prima operazione: GPIOB_MODER &= ~(3U << 18). La sequenza usa BIC con maschera 0x000c0000 e termina con STR a 0x0800002e. Dopo cinque stepi, PC = 0x08000030.

Seconda operazione: GPIOB_MODER |= (1U << 18). La sequenza usa ORR con maschera 0x00040000 e termina con STR a 0x0800003a. Dopo altri cinque stepi, PC = 0x0800003c.

Momento	GPIOB_MODER osservato	MODE9
Prima della configurazione	0xffffffebf	3: analogico
Dopo azzeramento dei due bit	0xfff3febf	0: input
Dopo impostazione del bit 18	0xfff7febf	1: output

Il passaggio intermedio per input deriva dalle due scritture separate del firmware. Per verificare PB9 interessa il campo MODE9; gli altri bit dell'intero registro appartengono ad altri pin.

12. ODR: l'istruzione che accende il LED

La riga GPIOB_ODR |= (1U << 9) corrisponde alla sequenza:

```
0x0800003c: ldr   r3, [pc, #20]
0x0800003e: ldr   r3, [r3, #0]
0x08000040: ldr   r2, [pc, #16]
0x08000042: orr.w r3, r3, #512
0x08000046: str   r3, [r2, #0]
```

La literal pool a 0x08000054 contiene 0x48000414, indirizzo di GPIOB_ODR. La maschera 512, cioè 0x200, imposta OD9. Dopo le prime quattro istruzioni, il valore da scrivere è preparato in r3; la STR a 0x08000046 trasferisce il valore al GPIO.

```
x/wx 0x48000414
p/x (*(unsigned int *)0x48000414 & 0x200)
```

L'accensione del LED verde dopo lo step della scrittura è stata confermata sulla scheda. Per OD9 la maschera attesa è 0x200; non è stato incollato nel resoconto l'output numerico finale di ODR, quindi questo valore è riportato come verifica attesa, non come trascrizione acquisita.

Il LED è active HIGH. Fermare la CPU dopo la scrittura non azzera ODR: il GPIO mantiene il livello impostato e il LED può restare acceso mentre il processore è fermo.

13. Breakpoint hardware

```
hbreak Reset_Handler
hbreak *main
info breakpoints
monitor reset halt
maintenance flush register-cache
continue
```

Nel laboratorio il primo continue ha prodotto:

```
Breakpoint 1, Reset_Handler () at src/startup.s:14
14  bl main
pc = 0x08000008, sp = 0x20010000, lr = 0xffffffff
```

Un secondo continue ha raggiunto:

```
Breakpoint 2, main () at src/main.c:8
8  {
pc = 0x08000010, sp = 0x20010000, lr = 0x0800000d
```

Il reset con halt può già arrestare la CPU sull'handler; osservare sempre PC e il messaggio di stop. hbreak richiede un breakpoint hardware. L'asterisco in *main seleziona l'indirizzo d'ingresso esatto, prima del prologo; break main può fermarsi sulla prima riga sorgente utile dopo il prologo.

14. Comandi essenziali e problemi incontrati

Comando	Uso
stepi	Esegue una singola istruzione macchina
nexti	Avanza di istruzione superando le chiamate
step / next	Avanzano a livello di sorgente
continue	Riprende fino a un evento di arresto
Ctrl-C	Interrompe l'esecuzione libera
info breakpoints	Elenca breakpoint e stato
delete breakpoints	Elimina tutti i breakpoint
monitor reset halt	Resetta e ferma il target tramite OpenOCD
maintenance flush register-cache	Invalida la cache registri di GDB

Problema 1: un blocco multilinea incollato è stato interpretato come un unico comando target. GDB ha tentato di risolvere un nome host che includeva le righe successive. Soluzione adottata: un comando per volta, con Invio dopo ogni riga.

Problema 2: add r7, sp, #0 è stato inserito nel prompt. È assembly del firmware, non un comando GDB. L'errore non ha fatto avanzare la CPU; stepi ha eseguito l'istruzione corretta.

Problema 3: valori di registro rimasti nella cache dopo monitor reset halt nella verifica preliminare. Soluzione verificata: maintenance flush register-cache prima della lettura di PC/SP.

Un Invio su una riga vuota può ripetere il comando GDB precedente: durante la sessione il messaggio di reset è comparso due volte. Evitare Invio aggiuntivi quando si desidera soltanto leggere l'output.

15. Chiusura della sessione e limiti

Per osservare il loop finale, usare continue e poi Ctrl-C. La CPU dovrebbe fermarsi nel while, mentre il LED mantiene il suo stato. Questa prova è proposta come esercizio conclusivo, non è inclusa negli output finali incollati dall'utente.

Per chiudere lasciando il firmware in esecuzione, inserire una riga alla volta e confermare l'eliminazione dei breakpoint se richiesto:

```
delete breakpoints
monitor reset run
detach
quit
```

Terminare OpenOCD con Ctrl-C nel primo terminale. Gli indirizzi delle istruzioni documentati appartengono alla build corrente: una modifica al sorgente, ai flag o alla toolchain può cambiarli. Rileggere il disassembly prima di usare indirizzi numerici come punti di arresto.

Lo startup rimane minimale: niente copia di .data, azzeramento di .bss o vector table completa. Non sono ancora stati studiati interrupt, fault handler o il Cortex-M0+. L'accesso a ODR resta read-modify-write. Questi limiti sono quelli del Project 01 e non sono stati risolti dal debugger.

16. Conoscenze acquisite e prossimo progetto

È stato collegato il comportamento del C alle istruzioni Thumb effettive, distinguendo indirizzo di un registro, contenuto del registro e valore temporaneo nei registri CPU. Sono stati osservati lo stack discendente e il link register, e usati breakpoint per raggiungere punti precisi del programma.

```
Reset -> vector table -> MSP e Reset_Handler
      -> BL main -> prologo e stack
      -> RCC.GPIOBEN -> clock GPIOB
      -> MODER.MODE9 = 01 -> PB9 output
      -> STR su ODR -> PB9 HIGH -> LED acceso
```

Il Project 03 introdurrà GPIOB_BSRR e il lampeggio con busy-loop. Il confronto con ODR permetterà di capire perché una scrittura set/reset evita il read-modify-write dell'uscita. Il debugger potrà essere usato per osservare il nuovo comportamento con lo stesso metodo.

17. Riferimenti e file del progetto

- ST RM0453, copia locale in docs: memory map, RCC e GPIO dello STM32WL5x.
- Documentazione del Project 01: docs/STM32WL55JC1_Bare_Metal_Documentation.pdf.
- Guida pratica: docs/project-02-debugging.md.
- Configurazione: openocd.cfg. Firmware: src/main.c, src/startup.s e linker.ld.
- OpenOCD, GDB and OpenOCD: https://openocd.org/doc/html/GDB-and-OpenOCD.html
- OpenOCD, target STM32WL: https://github.com/openocd-org/openocd/blob/master/tcl/target/stm32wlx.cfg
- GNU GDB, documentazione ufficiale: https://sourceware.org/gdb/current/onlinedocs/gdb.html/

Stato del laboratorio: connessione OK; reset e stack osservati; RCC e MODER verificati; LED acceso con single-step; breakpoint Reset_Handler e main verificati.